ABSTRACT

Title of Dissertation: TO
BE
DETERMINED

Kent Daniel O'Sullivan
Master of Science, 2023

Dissertation Directed by: Professor William Regli
Department of Computer Science

ABSTRACT HERE

TITLE HERE

by

Kent Daniel O'Sullivan

Dissertation submitted to the Faculty of the Graduate School of the

University of Maryland, College Park in partial fulfillment
of the requirements for the degree of
Master of Science
2023

Advisory Committee:
William Regli, Chair/Advisor
TBD
TBD

© Copyright by
Kent O'Sullivan
2023

Preface

If needed, PREFACE HERE.

Foreword

If needed, FOREWORD HERE.

Dedication

If needed, DEDICATION HERE.

Acknowledgments

ACKS HERE

Table of Contents

Preface ii

Foreword ii

Dedication ii

Acknowledgements ii

Table of Contents iii

List of Tables iv

List of Figures v

List of Abbreviations vi

Chapter 1: Literature Review 1

1.1 Graph Analysis In the real world 1

1.2 Graph Processing - Computational Challenges and Benefits 2

1.3 High Performance Computing 2

1.4 Benchmarking 4

1.5 Graph Benchmarking 6

1.6 Datasets and Generators for Graph Benchmarking 13

1.6.1 Problem Domains and Graph Algorithms 15

Bibliography 24

Bibliography 24

List of Tables

1.1 Summary of Graph Benchmark Datasets. STAT = Statistics, BFS = Breadth First Search, SSSP = Single Source Shortest Path, PR = PageRank, CC = Connected Components, BC = Betweenness Centrality, TC = Triangle Counting, CD = Community Detection, SGM = Sub Graph Matching, KGA = Knowledge Graph Analytics, ET = Execution Time, TEPS = Traversed Edges Per Second, LT = Load Time, SLS = Spatial Locality Score, Seq = Sequential, Par = Parallel/Distributed, CPU = Central Processing Unit, GPU = Graphics Processing Unit, DSA = Domain Specific Architecture 12

List of Figures

List of Abbreviations

DARPA Defense Advanced Research Projects Agency HIVE Hierarchical Identify Verify Exploit

Chapter 1: Literature Review

Plan for Lit Review

RQ: How do we benchmark knowledge graph operations across multiple hardware architectures?

1. *Why do we care about KG's in the first place?*
2. *Why do we care about Benchmarks?*
3. *What makes a good benchmark?*
4. *How have previous efforts benchmarked knowledge graphs?*
5. *What do we measure?*
6. *What data is out there?*
7. *What metrics do we use?*

1.1 Graph Analysis In the real world

Graph Analysis Use Cases? Establish why we care

1.2 Graph Processing - Computational Challenges and Benefits

Maybe about Graph Processing in general?

1.3 High Performance Computing

Dally, Keckler, and Kirk's 2021 historical review of Graphic Processing Unit (GPU) development highlights the duality of the relationship between enabling hardware and the applications that use it. They explore the close relationship between Machine Learning applications and GPUs. Their discussion suggests that while the availability of GPUs enabled more machine learning applications to be built, the reciprocal demand for higher-performance machine learning models drives the development of improved GPUs. The salient question is whether hardware optimized for graph processing will have the same impact in the data analytics domain that GPUs had on scientific computing [1].

The lengthy 2023 report by Mutlu and their peers from the SAFARI research group is a response to the perceived limitations of the dominant processor-centric computing paradigm. The authors characterize the processing-centric paradigm as one where large and dispersed data are moved to a central processing unit (or GPU etc.) to be processed and returned to memory. They argue that the processor-centric architecture is inefficient and that rather than being compute-bound, most modern applications are memory-bound as a result. Their analysis finds that up to 62 percent of all power used by computers is just the action of moving data to and from memory. Their response is a processing in memory (PIM) paradigm. PIM is an old idea, but they argue is becoming viable with the emergence of new hardware technology like 3D chips. They define

two methods of PIM: Processing Using Memory (PUM) and Processing Near Memory (PNM). PUM acknowledges that many of the simplest functions like adding scalar values to entire rows of memory, or initializing large blocks a simple enough primitive options that they can be performed by the memory without having to be mapped to the CPU. PNM recognizes that emerging 3D chips have small logic controllers that can be leveraged for simple decentralized processing actions. Their paper makes extensive references to Graph Processing. They identify that a driving cause of poor graph processing performance is the random memory accesses that result from sparse adjacency matrix representations. A second reason for poor performance is that the actual processing of items pulled out from memory is trivial and completed quickly, exacerbating the effects of memory access latency. They designed an architecture TESSERACT which as described appears to be a competitor to PiUMA. Regarding GPUs, they assert that they hide long latencies of memory accesses by interleaving arithmetic and logic operations. They present DAMOV, their framework for measuring memory-boundedness, identifying common culprits such as cache misses, cache coherence traffic, and long queueing latencies. They use intel V-TUNE for the profiling aspect of DAMOV and then implement locality-based clustering to characterize the spatial and temporal clustering features of applications under test. Overall this work motivates the need for specialized architectures to improve performance and proposed Processing-In-Memory as a paradigm to address the limitations of processor-centric models, including specifically for graph processing. They describe TESSERACT as a possible competitor to PiUMA and DAMOV as a profiling and simulation suite [2].

1.4 Benchmarking

A 2005 Study by Weinberg et al. examines the measurement of spatial and temporal locality in high-performance computing. They define spatial locality as memory addresses that are located close to each other, and temporal locality as the same memory addresses being accessed repeatedly over time. They conduct their measurements by instrumenting their code using the MEMSIM platform and using it to generate a spatial and temporal locality index with parameters L and K. L is a measurement of the stride size, and K is a measure of the randomness of access. They present these metrics despite earlier warnings in their work about the reductive effects of generating an index to represent locality. Their analysis suggests that a lack of locality degrades the execution time of applications and so it is interesting that neither Graph500 nor the newer graph benchmark suites account for the loading or structuring of graphs in memory in their measurement. Characterizing the shape of graphs in memory appears to be an under-explored dimension of the benchmarking space [3].

In 2009 Adhianto et. al. from Rice University released the initial build of the High Performance Computing Toolkit (HPCToolkit). The toolkit aims to profile the performance of code on high-performance systems without needing to instrument the code, reducing the overheads imposed on the system under test. Their approach and toolkit exemplify the measurement of performance without instrumentation, demonstrating the feasibility of profiling code without intrinsically degrading its performance [4].

A 2010 description of how Google implements system profiling across its data warehouses by Ren et al. focuses on continuous monitoring rather than benchmarking. Their experience argues that sampling binaries during execution rather than fully instrumenting at compile time

is a superior approach that reduces memory usage and execution time. They sample events, which can include clock cycles, L1 and L2 cache misses, and branch mispredictions. Their work provides a precedent for using profiling to compare different hardware implementations of the same application, supporting our evaluation of Graph Algorithms across CPU, GPU, and PiUMA for the HIVE project. There appears to be a gap in defining what a standard 'profile' is for a graph algorithm. Determining what a standard 'profile' is, and further determining a method to effectively visualize memory accesses for graph applications by time and locality will be a prosperous avenue of further research [5].

The 2015 paper "Profiling a Warehouse-scale Computer" from Kanev et al. provides insights into limitations and bottlenecks that computing at large scale experiences. While their work is not directly relevant to benchmarking graph algorithms, elements of their methodology are useful. For example, they conduct their profiling by randomly sampling from active machines, then levering the Linux Perf suite to collect data. They then tag the observations to link them to the code generating the observed behavior and load the results into the Dremel database for analysis. They use the Top-Down profiling approach. The Top-Down approach uses the micro-operation queue to classify operations into one of four categories. The patterns of micro-operation occurrences drive the characterization of system behavior. They assert the canonical approach to determining instruction set size is to simulate and then look for the elbow point in the simulation where cache misses drop to zero, and offer an alternative method that samples the real system instead. They have two interesting findings relevant to the HIVE problem. First, the main reason that they see back-end micro-operation slots being created is to serve data cache requirements. Second, 95 percent of the systems that they analyze use 31 percent or less of their memory bandwidth. Taken together, we can surmise that the size of the data being read from and written

to memory is not the bottleneck, it is the latency in waiting for the accesses to occur. In graph processing, because the memory access patterns are not localized, and the processing operations are very simple the latency effect will be exacerbated. Finally, they identify that simultaneous multi-threading is a noted mechanism to improve overall performance assuming that there is a diverse cause of bottlenecks. It is not clear whether the parallelizing of graph algorithms will improve or degrade memory latency in graph processing [6].

1.5 Graph Benchmarking

In 2010 Richard Murphy and his team from Sandia National Labs introduced the Graph500 dataset as a corollary to the Top500 dataset that tests floating point operations per second (FLOPS) on high-performance computers. Graph500's initial goal focuses on creating benchmarks for the search, optimization, and edge operations of graph kernels (i.e. types of tasks). They specify the parameters to be used in RMAT to generate synthetic graphs but are unclear on the actual metric they are benchmarking against. In context, it appears to be temporal (i.e. time to complete an operation), which they note in their initial experiments is already limited at large scales. Graph 500 is a useful standard but relies on synthetically generated data and focuses narrowly on a few problems, with simplistic metrics [7].

In 2013 Angles et al. propose their benchmark for social database systems. Their approach creates micro-benchmarks-based query primitives for social networks they identify as (1) Selection, (2) Adjacency, (3) Reachability, and (4) Summarization. They present a generator to create synthetic datasets, evolving the R-MAT algorithm to create a 'streaming' dataset by simulating the R-MAT recursion, storing the distribution of edges, and then constructing the graph after the

fact. Their benchmark compares SQL, Graph, and RDF (Knowledge Graph) implementations on similar queries. Rather than implement the algorithms themselves, they use the query languages of DBMS like PostgreSQL, Neo4J, and RDF-3X. They measure both graph load time, (calculating objects per second by dividing total load time by total node+edge counts) and execution time. They find that Reachability queries are the most computationally intensive, and unviable on non-native graph structures, like the relational data stores. Though their benchmark seems to lack some formality, it does offer the ability to support streaming data, and knowledge graphs and gives us primitive operations to form the core of our exploration of knowledge graphs. They highlight that the cost of translating between URIs and internal representations in KGs can be expensive, particularly for simple queries. Though not evaluated here, their graph generator offers a potential path to transform a static graph into a streaming graph [8].

The 2015 Graphalytics Benchmark from Capotă et al. aims to produce consistent reporting on graph workloads between all combinations of algorithms, datasets, and platforms. It primarily targets distributed and parallel implementations. Their paper claims (but provides no substantive evidence or discussion of) support for evaluating algorithms run on GPUs and knowledge graph analytics. They assert that a good benchmark suite must balance using real-world datasets with designing specific problems to stress the known choke points. After highlighting that the use of real datasets is essential for credibility, they explain how they create their synthetic datasets. Specifically for graph workloads, they identify (1) Excessive Network Utilization, (2) Large Memory Footprints, (3) Poor Access Locality, and (4) Skewed Execution Intensity. They characterize their datasets by Vertex and Edge Count, Global and Average Cluster Coefficients, and Assortativity (The assortativity coefficient is the Pearson correlation coefficient of degree between pairs of linked nodes). They also examine the degree distribution by the goodness of

fit to several known distributions. The Graphalytics suite supports five algorithms: (1) General Stats, (2) Breadth First Search, (3) Connected Components, (4) Community Detection, and (5) Graph Evolution. They do not measure or report the time taken to extract, transform, and load the graph into memory, or any detail about how it is structured in memory. The Benchmark Suite measures execution time and traversed edges per second (calculated by dividing the execution time by the total number of edges - it is not clear if this will handle algorithms that revisit edges multiple times, and a more robust approach is required). They assert that software engineering best practices should be applied to any benchmarking effort, and use static code analysis and formal change management to provide quality assurance for their system, justifying our use of TDD in approaching our solution [9].

Beamer et al. from UC Berkeley introduce the GAP Graph Benchmark in their 2017 paper in response to their perceived shortcomings with the Graph500 benchmark. The authors assert that a benchmark suite should use real and diverse data wherever possible, and when synthetic data is used it must be standardized. They note among the many views of a benchmark suite, that GAP is suited to comparing the performance of hardware on common graph problems. They provide reference implementations that are tested across multiple compilers. While the GAP Benchmark paper refers to spatial locality, they do not clearly explain how they measure the effect of execution time or traversed edges per second. They address variation in execution time caused by differing start nodes by repeatedly running workloads with different start points. The benchmark is designed to be wider-ranging than Graph500, and as a result, has reference workloads for Breadth First Search, Single Source Shortest Path, Page Rank, Connected Components, Betweenness Centrality, and Triangle Counting. While Connected components and betweenness centrality have some relation to community detection, and triangle counting is related to

subgraph matching none are exact matches for the problem domain that HIVE is specifically pursuing. There is no mention at all of knowledge graph analytics. Given that overall system performance includes the loading and storage in memory of Graphs it appears to be a gap that these parts of the process are not measured. An examination of how a graph is stored in memory affects its spatial locality during execution appears to be under-explored and so may be worth analyzing further [10].

The blog is written on behalf of Memgraph and is an explanation of their benchmarking architecture. They describe the Python-based experiment framework that sits atop the experiment architecture's lower-level files. Of note, they highlight that Docker imposes a significant performance overhead on the experiments. For merging with QUICC, the impact of docker is a consideration to declare up-front. Their benchmark process includes measurements of creation time, as well as restoration from snapshot - it is more focused on a production grad GDBMS than the graphs themselves. They deliberately measure cold-start performance to get the 'worst-case' scenario for their graph benchmark. Overall it provides a useful framework to consider for the design of a benchmark suite [11].

Written on behalf of Memgraph (a Neo4J competitor), Javor's blog post summarizes a benchmarking effort made between the leading commercial graph database, Neo4J, and Memgraph's graph database. While there is inherent bias, it is useful to note how they approached the benchmark. Their metrics covered temporal (latency in MS, Queries per Second) and Memory (Peak memory usage). They ran three types of 'business' workloads - single, mixed, and realistic. Single repeat the same query, mixed varies between several query types, and realistic varies according to usage patterns seen in the real world. One of the criticisms of the benchmark was providing the data and queries written in Cypher (the graph query language), however,

that sets it up for a more realistic use case. Few people will just directly import masses of data into their database directly. Importantly for HIVE, this experimental design consideration will reflect the reality of a 'streaming' data effect. Additionally, it highlights the impact that caching can have on performance, and so to get a 'worst' case performance, they run the queries on a 'cold' start. The most load-intensive query they run in their workloads is a 'k-hop' or 'explosion' query, which is effectively a BFS of the graph to a specified depth. The benchmark does make the dataset available, but the post does not discuss it in any real detail. It is a useful post, and worth chasing down scientific publications to validate their design choices. The benchmark is here: `https://memgraph.com/benchgraph/` and the code is here: `https://github.com/memgraph/memgraph/tree/master/tests/mgbench` [12]

Memgraph provides the methodology (and code) for their BenchGraph graph database benchmark on their GitHub. It describes the methodology and several of their design choices. First, they identify the primitive operations they test as: "write", "read", "update", "aggregate" and "analyze". They use three datasets - a small medium and large. Each of them are social-media style scale-free graph. They acknowledge the need to support different hardware types in testing and to that end provide a framework to test two different hardware platforms. While trivial (and in this case dependent on what is capable of running the GDBMS under test) it highlights the need for cross-hardware tests. The benchmark itself overall is a good starting point for designing a graph dataset - it includes datasets of varying size (but not type), reference queries, reference code, and support for multiple hardware platforms. It is limited in that it focuses on GDBMS, rather than graphs in the abstract sense. A weakness of their benchmark is the use of the Neo4J bolt protocol for loading the data, which is known to impose significant latency on data. [13]

Max De Marzi's 2023 blog post is a review of the graph benchmark released by Memgraph. The blog is a biting criticism of how Memgraph conducted the tests and possible bias in their reporting. In particular, Max criticizes Memgraph for developing a bespoke system for measuring performance rather than using the industry standard "Gatling" to control the tests. Gatling is commercial load-testing software. Overall it is a vitriolic attack on Memgraph, and some of the criticisms of Max are addressed in the benchmark itself (e.g. why they only report the 99th percentile results). Based on these criticisms, it is evident that any new benchmark needs to be very clear about what it brings to the table that previously did not exist [**?**].

The consulting company McKnight produced a benchmark report on the performance of Stardog, at the request of Stardog. While there is inherent bias in the study, it warrants some attention. First, their approach to data storage replicated a 'real world' use case, with the trillion edges they note in the title being spread between native RDF in Stardog and other non-triple stores. Their data is generated with the Berlin SPARQL Benchmark (BSBM). Their measured metric is query latency, but given that their test server had 1TB of RAM, it indicates that they intended to hold most of the graph in memory on a single system, this will reduce the impact of network latency on their results. Their reference queries are derived from the BSBM and consist of simple 'explore' queries, ignoring the more computationally challenging inference-based queries that distinguish knowledge graphs from 'normal' graphs. Any meaningful knowledge graph benchmark needs to have inference benchmarks (though admittedly, many outsource their reasoner). Their choice of the BSBM is concerning because it is around 20 years old at this point, and not likely to be reflective of the scale-free data we frequently see associated with graph processing now [**?**].

Benchmark Suite	Algorithm										Metrics				Imp		Ar	
	STATS	BFS	SSSP	PR	CC	BC	TC	CD	SGM	KGA	ET	TEPS	LT	SLS	Seq	Par	CPU	GI
Graph500 [2010] [7]		X									X				X		X	
Social Benchmark [2013] [8]	X									X	X		X		X		X	
Graphalytics [2015] [9]	X	X			X			X		?	X	X			X	X	X	
GAP [2017] [10]		X	X	X	X	X	X				X				X		X	

Table 1.1: Summary of Graph Benchmark Datasets.
STAT = Statistics, BFS = Breadth First Search, SSSP = Single Source Shortest Path, PR = PageRank, CC = Connected Components, BC = Betweenness Centrality, TC = Triangle Counting, CD = Community Detection, SGM = Sub Graph Matching, KGA = Knowledge Graph Analytics, ET = Execution Time, TEPS = Traversed Edges Per Second, LT = Load Time, SLS = Spatial Locality Score, Seq = Sequential, Par = Parallel/Distributed, CPU = Central Processing Unit, GPU = Graphics Processing Unit, DSA = Domain Specific Architecture

1.6 Datasets and Generators for Graph Benchmarking

Chakrabarti, Zhan, and Faloutsos' 2004 Recursive Matrix (R-MAT) remains a de facto standard in synthetic graph generation. R-MAT works by a simple mechanism, accepting parameters a, b, c, and d, which are numbers between 0 and 1, that must add to 1. Each parameter reflects the probability that a node will be placed in a given quadrant of an adjacency matrix. The placement is done recursively, with each quadrant being subdivided into 4 regions until the base case is reached and the node is assigned. They briefly explain how one could estimate the values of a, b, c, and d but it is not clear whether a tool that can analyze a graph and estimate the values of A, B C, and D is available, or proven to work. If it does not, this would be a useful contribution. If it does, it will be worth examining their method to enable the generation of realistic datasets for the PiUMA experimentation [14].

Lancichinetti et al. developed the LFR algorithm for graph generation in 2008 to address the problem they observed that most benchmarking efforts use small, homogeneous Erdős-Rényi (random) graphs, which are not reflective of real-world benchmarks. They assert that the ideal graph for evaluating community detection is a real-world graph with ground-truth community labels, but acknowledge that only trivially sized graphs exist meeting these criteria. They present an iterative algorithm for generating heterogeneous communities that accepts the following parameters: γ [degree exponent], β [community size exponent], n [number of nodes], $< k >$ [average degree], and μ [the mixing parameter]. The algorithm works by generating n nodes with degrees sampled from a power law distribution (determined by γ). The nodes are randomly assigned to communities, sharing $1 - \mu$ links with their community, and μ links with nodes outside their community. In each iteration, nodes are randomly moved to other communities - if

the community becomes too big (determined by β), it removes some nodes and re-assigns them. The algorithm continues until convergence. They do not provide a formal complexity analysis of their algorithm, but claim that it 'performs in reasonable time'. Their evaluation metric is the 'Normalized Mutual Information' score. They identify a few pain points in the shape of the data that make it difficult for community detection algorithms to find 'accurate' solutions. First, When the mixing parameter $\mu <= 0.5$, the communities become 'weakly' defined relative to the graph. Second, small communities are difficult to detect with modularity optimization approaches and third, they observe that graphs with a smaller average degree (i.e. sparser graphs) perform less well. Fourth, They find that the closer communities are in size (larger β) the better community detection algorithms perform [15].

In 2019 Maekawa et al. developed the acMark graph generator to allow for fine-grain control of communities that emerge in synthetic attributed (labeled) graphs. They develop acMark because the most effective community graph generators do not generate labeled graphs, existing ML generation approaches don't scale well and existing statistical approaches don't offer fine-grained control. acMark works by accepting parameters from the user that describe the latent communities within the graph, and then uses these parameters to iteratively generate communities with a greedy algorithm. Based on their presented documentation, the generator looks to be an effective tool for generating community-based graphs with labels. It would also be applicable in generating knowledge graphs. One unclear factor is how this differs in intent from triple-generators, and whether a triple-generation approach would be more effective for generating attributed graphs with community structures. The real challenge here is figuring out how to determine the characteristics of a real-world graph and then using those characteristics to replicate the graph [16].

1.6.1 Problem Domains and Graph Algorithms

1.6.1.1 Community Detection

In 2004, Reichardt and Bornholdt used a physical systems view of graphs to develop a community detection approach. Their approach involved characterizing the graph as the Potts Spin-Glass Hamiltonian of a system, characterizing the 'goodness' of the community as the 'energy' within the graph. They use Monte Carlo simulation to derive the 'base', or 'global' energy state of the graph. Then, they project the system as a spin glass - a physical system that derives its energy state from the arrangement of its components. They 'spin' the graphs (i.e. alter the connections in possible communities) and search for local minima in energy that are better than the global configuration. Where true, this indicates a 'better' community structure than the 'global' exists, suggesting it is more densely connected than the broader graph. They present their math, and claim that is it 'extremely fast' compared to earlier approaches, but do not do a formal complexity analysis. [KENT: I attempted to interpret it and got $\mathcal{O}(M + Q^2)$ where M is the number of edges, and Q is the number of communities expected, and assuming that the calculation time for each spin state is proportional to Q. - I am not confident]. A major weakness of this approach is that the number of communities must be estimated in advance. If the estimate is too small, not all communities will be found. If too large, it will increase the amount of spins and degrade performance. They identify a pain point in their approach as being when communities become 'fuzzier' when they are not separated from the graph and have a similar number of outward links as they have internal links. The data they test on is all trivially small and has a homogenous community size [17].

In 2005, Muff et al. extended the idea of a global modularity score (Q) with the concept of local modularity (LQ). They claim that modularity optimization is the superior approach to community detection because it needs no a-priori information about the structure of the communities, but assert the need for a local modularity score is driven by communities being inherently local, and unsuited for a global view. To emphasize their point about false global views, they explain how Q is calculated based on an Erdős-Rényi graph distribution, and so it treats the likelihood of a link between all nodes as equally likely, even if not all links are possible. As an example, they show how a graph of school students that has densely connected classes, loosely connected year groups of several classes, very loose connections between classes +- 1 year, and no connections +-2 or more years from each other will fail in modularity optimization. Q will aggregate on the global 'year' view rather than the smaller (and better) 'class' view. LQ is most useful (discriminative compared to Q) when the outer link distribution for communities is very weak. That is, when dense communities are loosely connected, LQ is a better scoring system. The authors assert that LQ is not a replacement for Q, but should be used in concert with it. Overall it appears to add limited value to the calculation of Q apart from some edge cases it appears you need a-priori knowledge of existence to search for [18].

In 2006 Reichardt and Bornholdt sought to formalize the conduct of community detection. They had noticed that there was a lack of formal rigor in the literature and sought to rectify the gap. They contribute a taxonomic framework of 'clustering' versus 'partitioning' approaches to community detection. They show that while hierarchy frequently does exist in communities, it cannot be assumed and highlight how this assumption can break some algorithms (particularly partitioning ones). They emphasize that an Erdős-Rényi graph will typically present a modularity score indicating the presence of a community, and argue that to be a true 'community' a modu-

larity score must exceed the modularity of an Erdős-Rényi graph of equivalent size. They also contribute a method to detect a single community from a given start node, aiming to remove the need to process an entire graph when not required for the user query. They identify a few pain points: first is that divisive approaches using simulated annealing tend to over-separate graphs, whereas agglomerative approaches tend to over-group. They also identify that Sparse Erdős-Rényi graphs tend to have lots of small communities, whereas dense Erdős-Rényi graphs have few large communities. Specific failure modes that they identify include: for Recursive Bisection approaches - when there are 3 equally sized communities, at least one of them will be mistakenly grouped with another, and will usually be split across both. Greedy Agglomerative approaches fail when two cliques are connected by nodes of low degree compared to the clique. Finally, they note that there can be lurking reasons for overlap in community membership that are not explicitly represented in the graph. That lurking factor problem emphasizes the importance of a labeled graph and highlights what a KG view of the graph contributes to the existing literature [19].

Fortunato and Barthélemy's 2007 work explored whether a resolution limit exists in modularity optimization. Their study was driven by a perceived lack of formal rigor in the application of Modularity Optimization to Community Detection Problems and the lack of baseline understanding of performance. Starting with their definition of community as 'a subgraph whose members are more tightly connected than the broader graph', the paper is a theoretical analysis of the problem that uses trivial and extreme data to derive the conclusion that modularity is not scale-independent. That is, a scaling factor exists where a community with a density of internal links of the order of the square root of 2 times the number of links in the graph will not be resolved by modularity optimization. These 'loosely connected' communities are instead combined. They identify two particular pain points in the shape of the data that impact the accuracy of the mod-

ularity optimization. First, is when small communities coexist with large ones (heterogeneous communities). The second occurs when the community size distribution is broad. Towards a data generator, they note that a graph with 'perfect' modularity is a closed ring with each node only connected to its two closest neighbors. Or in a hierarchical view, communities that are only connected through single bridging nodes [20].

In 2007 Reichardt and Bornholdt extended their prior work on community detection, seeking to determine if graph partitioning approaches are usable with relaxed constraints towards community detection. They note that prior work in graph partitioning has shown it to be an NP-complete problem. To relax the constraints, they frame it as a replica method of a Hamiltonian Spin-Glass. The idea of a Hamiltonian spin-glass is explained in their 2004 work, and the 'replica' method essentially means that they duplicate the system under test multiple times and average the results. They test their approach on Erdős-Rényi graphs and Scale-Free graphs, focusing on Bi-Partitioning but arguing it would generalize to n-partitioning. Their work does not include an explicit complexity analysis. They generate their data with the Molloy-Reed Algorithm [Kent - I cannot find this]. Their analysis shows that the performance of their approach slows as the underlying graph gets sparser [21].

In 2008 Blondel et al characterized what in time becomes known as the Louvain Algorithm. The Louvain Algorithm is a heuristic algorithm to approximate community detection in graphs. It uses an interactive 2-step algorithm to maximize the modularity score of communities hierarchically. The algorithm first assigns each node in the graph to a different community and calculates the improvement in modularity scores. Then it creates a new instance of the graph where changes to the communities increase the modulatiry score. The algorithm terminates when no changes to the modularity score result from an iteration. They define community detection algorithms as

belonging to one of three categories: (1) Divisive, (2) Agglomerative, and (3) Optimization. The Louvain Algorithm is agglomerative. They assert that their algorithm is bound by storage, not computation. Assuming that they mean memory when they write storage, it conforms to the expected behavior of graph platforms motivating the HIVE program. They observe that despite a minimal effect on the ending modularity scores, the starting node (and order of execution) has a varying effect on runtime. They were not able to determine why runtime is affected by the start node. They assert a linear complexity for their algorithm, but do not present a formal proof, but show empirically that they achieve superior modularity and runtime results compared to the other algorithms at the time. Runtime and Modularity appear to be suitable metrics for a community detection benchmark. Finally, they postulate further runtime improvements by introducing additional heuristics, like a 'good enough threshold for modularity [22].

In their 2017 paper, Cavallari et al extend the idea of graph embeddings at the node level to the community level, presenting the COM-E algorithm. They implement COM-E as a closed-loop, with an initial community detection partition feeding the creation of a community embedding which in turn improves the node embeddings and is fed back into improving community detection. They use spectral clustering as their initialization, in approaches reminiscent of earlier attempts to improve community detection by using the Kernighan-Lin algorithm to fine-tune the initial partitions generated with spectral partitioning, or spectral clustering. They assert that their method has a complexity of O(—V—γl+—V—+T1$\times$(T2—V—K+K+—E—+—V—γl+—V—K)) which they claim is linear to the graph and reduces to O(n+m). They evaluate the Karate Club, BlogCatalog, Flickr, Wikipedia, and DBLP datasets. While they note improvement over other node-embedding methods, they use the metrics of Conductance (ratio between the number of edges leaving the community and the number in the community - smaller numbers are better)

and Normalized Mutual Information (Closeness between predicted communities and ground-truth labels - higher is better.) which make comparisons to non-embedding methods that use Modularity difficult. Their empirical results suggest that the 'accuracy' of community detection struggles on datasets with more communities, higher edge density, and multiple labels per node (i.e. overlapping communities). We can infer based on the performance of the DBLP dataset that it performs best on clique-like graphs, and most poorly on 'fuzzy' communities. They present no empirical timing information. They note that node embedding approaches quickly become memory intensive because of the size of sparse graph matricies [23].

Tandon et. al. produced a survey paper on the use of Graph Embedding techniques for Community Detection. They taxonomize the existing approaches into those using matrix-based methods, and those using random-walk-based methods. They use the LFR benchmark datasets to compare the graph-embedding approaches to popular non-embedding methods (like Louvain). They note that in general embedding approaches are competitive on smaller graphs, but are outstripped by the conventional clustering methods on larger graphs. Moreover, they note that the requirement to derive optimal parameters to run these embedding algorithms is negative when compared to the existing non-embedding approaches. Overall, they find that the use of graph embeddings offers no real advantage over the existing modularity-based clustering approaches [24].

White et al presented two spectral clustering algorithms in 2005 as extensions of earlier work in spectral clustering specifically to community detection. They propose two Algorithms, Spectral1 and Spectral2. Spectral 1 focuses on optimizing the modularity score of their communities under detection, sacrificing speed. It works by first projecting the graph into Euclidian space, then using k-means clustering to create partitions (beginning with k = 2 and ending at k = the user parameter K of maximum communities). Spectral2 speeds up this process by using

greedy bisection to speed up the search for the optimal k ¡ K, at the cost of finding a sub-optimal modularity score. The expected complexity of both algorithms is given as 'roughly linear' under optimal conditions in a sparse graph where $n \propto m$, and so we approximate it as $\mathcal{O}(n \log n)$. The worst case is $\mathcal{O}(mkh + nk^2 + k^2h + nk^2e)$, where h is the number of iterations to convergence for generating eigenvectors, k is the number of communities and e is the number of iterations that k-means clustering is being run for. The worst case is expected to occur when the shape of the data has completely imbalanced community sizes and the largest community must be split each time (i.e. forcing a brute-force search of the possible K communities). An obvious drawback to this approach is the requirement for an a-priori estimate of the number of communities K, which has a direct impact on the runtime of the algorithms. They empirically evaluate their algorithms on the Wordnet Dataset, NIPS co-authorship dataset, and the American College Football dataset [25].

Ng's foundational work on Spectral Clustering from 2001 provides an introduction to spectral clustering techniques, as an alternative to spectral graph partitioning. In essence, their algorithm uses the k-largest eigenvectors to form approximately optimal clusters and then applies k-means clustering to tighten up the clusters. The method is reminiscent of the approach to spectral partitioning which leverages the Kernighan-Lin algorithm to 'fine-tune' the approximately optimal partitions of a graph. The 'ideal' data for this algorithm is distinct clique-like clusters all infinitely far apart. We can deduce then that the worst-case for this approach includes fuzzy, overlapping, and dense communities. They provide no formal complexity analysis and only generally speak about experimental results citeNg2001.

The Leiden algorithm is broadly heralded as the anointed successor to the Louvain algorithm. The 2019 work by Traag et al. focuses on addressing a problem observed in Louvain communities. They show that the aggregation steps of the Louvain algorithm can result in discon-

nected and weakly-connected communities. They do not give a formal complexity estimate for the Leiden Algorithm, so we derive it from their supplemental materials to be $\mathcal{O}(n^2)$ in the worst case and perform at least as well as Louvain in the average case - we assign the same asymptotic average case as $\mathcal{O}(n \log n)$. Empirically, they tend to execute more quickly than Louvain in the average case because they only traverse the nodes they need at each iteration rather than the entire adjacency matrix as Louvain does. The difference in execution time becomes pronounced on larger, showing an order of magnitude improvement to execution time when the graph size is in the tens of millions of nodes. The worst shape of the data for Leiden mirrors Louvain, where the mixing parameter μ is high, which in real-graph terms is when the graph itself is dense, and the communities are 'fuzzy' [26].

Family	Algorithm	Average Complexity	Worst-Case Complexity	
Graph Partitioning	Kernighan-Lin	$\mathcal{O}(n^3)$		
	Multi-Level Recursive Bisection	$\mathcal{O}(n^3)$		
	Metis	$\mathcal{O}(n^3)$		
	Recursive Spectral Bisection	$\mathcal{O}(n^3)$		
Graph Clustering	Ng's Algorithm [27]		$\mathcal{O}(n^3)$ [25]	
	Spectral_1 [25]	$\mathcal{O}(mkh + nk^2 + k^2h + nk^2e)$	$\mathcal{O}(mkh + nk^2 + k^2h + nk^2e)$	
	Spectral_2 [25]	$\mathcal{O}(n \log n)$	$\mathcal{O}(mkh + nk^2 + k^2h + nk^2e)$	Skewed Comm
	K-Means on Graph Embeddings	$\mathcal{O}(n * k * i * d)$		
	Hierarchical Agglomerative Clustering	$\mathcal{O}(n^3)$		
	DBSCAN	$\mathcal{O}(n^2)$		
	Louvain [22]	$\mathcal{O}(n \log n)$		
	Leiden [26]	$\mathcal{O}(n \log n)$	$\mathcal{O}(n^2)$	
Modularity Optimization	Louvain [22]	$\mathcal{O}(n \log n)$		
	Girvan-Newman	$\mathcal{O}(m^2 * n)$		
	Spectral Clustering	$\mathcal{O}(n^3)$		
	Greedy Optimization	$\mathcal{O}(n^2)$		
	Label Propagation	$\mathcal{O}(n)$		
	Hamiltoninan [Potts-Fuzzy] [17]		$\mathcal{O}(m + q)$	
	Single-Community Hamiltoninan [19]		Need to Calc	Distin
	Hamiltoninan [Replica] [21]		Need to Calc	
Graph Embedding	Deep-Walk		$\mathcal{O}(n \log n)$ [23]	
	LINE		$\mathcal{O}(\alpha m)$ [23]	
	Node2Vec		$\mathcal{O}(n \log n + \alpha n^2)$ [23]	
	Com-E [23]		$\mathcal{O}(n + m)$	

Bibliography

[1] William J Dally, Stephen W Keckler, and David B Kirk. Evolution of the graphics processing unit (gpu). *IEEE Micro*, 41(6):42–51, 2021.

[2] Onur Mutlu, Saugata Ghose, Juan Gómez-Luna, and Rachata Ausavarungnirun. *A Modern Primer on Processing in Memory*, pages 171–243. Springer Nature Singapore, Singapore, 2023.

[3] J. Weinberg, M.O. McCracken, E. Strohmaier, and A. Snavely. Quantifying locality in the memory access patterns of hpc applications. In *SC '05: Proceedings of the 2005 ACM/IEEE Conference on Supercomputing*, pages 50–50, 2005.

[4] L. Adhianto, S. Banerjee, M. Fagan, M. Krentel, G. Marin, J. Mellor-Crummey, and N. R. Tallent. Hpctoolkit: tools for performance analysis of optimized parallel programs. *Concurrency and Computation: Practice and Experience*, 22(6):685–701, 2010.

[5] Gang Ren, Eric Tune, Tipp Moseley, Yixin Shi, Silvius Rus, and Robert Hundt. Google-wide profiling: A continuous profiling infrastructure for data centers. *IEEE micro*, 30(4):65–79, 2010.

[6] Svilen Kanev, Juan Pablo Darago, Kim Hazelwood, Parthasarathy Ranganathan, Tipp Moseley, Gu-Yeon Wei, and David Brooks. Profiling a warehouse-scale computer. In *Proceedings of the 42nd Annual International Symposium on Computer Architecture*, pages 158–169, 2015.

[7] Richard C Murphy, Kyle B Wheeler, Brian W Barrett, and James A Ang. Introducing the graph 500. *Cray Users Group (CUG)*, 19:45–74, 2010.

[8] Renzo Angles, Arnau Prat-Pérez, David Dominguez-Sal, and Josep-Lluis Larriba-Pey. Benchmarking database systems for social network applications. In *First International Workshop on Graph Data Management Experiences and Systems*, pages 1–7, 2013.

[9] Mihai Capotă, Tim Hegeman, Alexandru Iosup, Arnau Prat-Pérez, Orri Erling, and Peter Boncz. Graphalytics: A big data benchmark for graph-processing platforms. In *Proceedings of the GRADES'15*, pages 1–6. 2015.

[10] Scott Beamer, Krste Asanović, and David Patterson. The gap benchmark suite. *arXiv preprint arXiv:1508.03619*, 2017.

[11] Ante Javor. Memgraph vs. neo4j: A performance comparison. blog, Apr 2022. Accessed 04 Sep 23.

[12] Ante Javor. Introduction to benchgraph and its architecture. blog, Apr 2023. Accessed 04 Sep 23.

[13] Memgraph. Benchgraph methodology. Github Readme, May 2023. Commit: ab38161cd2b1b9eaeda952eb51d2167727fdd447; Accessed 04 Sep 23.

[14] Deepayan Chakrabarti, Yiping Zhan, and Christos Faloutsos. R-mat: A recursive model for graph mining. In *Proceedings of the 2004 SIAM International Conference on Data Mining*, pages 442–446. SIAM, 2004.

[15] Andrea Lancichinetti, Santo Fortunato, and Filippo Radicchi. Benchmark graphs for testing community detection algorithms. *Physical review E*, 78(4):046110, 2008.

[16] Seiji Maekawa, Jianpeng Zhang, George Fletcher, and Makoto Onizuka. General generator for attributed graphs with community structure. In *Proceeding of the ECML/PKDD graph embedding and mining workshop*, pages 1–5, 2019.

[17] Jörg Reichardt and Stefan Bornholdt. Detecting fuzzy community structures in complex networks with a potts model. *Physical review letters*, 93(21):218701, 2004.

[18] Stefanie Muff, Francesco Rao, and Amedeo Caflisch. Local modularity measure for network clusterizations. *Physical Review E*, 72(5):056107, 2005.

[19] Jörg Reichardt and Stefan Bornholdt. Statistical mechanics of community detection. *Physical review E*, 74(1):016110, 2006.

[20] Santo Fortunato and Marc Barthelemy. Resolution limit in community detection. *Proceedings of the national academy of sciences*, 104(1):36–41, 2007.

[21] Jörg Reichardt and Stefan Bornholdt. Partitioning and modularity of graphs with arbitrary degree distribution. *Physical Review E*, 76(1):015102, 2007.

[22] Vincent D Blondel, Jean-Loup Guillaume, Renaud Lambiotte, and Etienne Lefebvre. Fast unfolding of communities in large networks. *Journal of statistical mechanics: theory and experiment*, 2008(10):P10008, 2008.

[23] Sandro Cavallari, Vincent W Zheng, Hongyun Cai, Kevin Chen-Chuan Chang, and Erik Cambria. Learning community embedding with community detection and node embedding on graphs. In *Proceedings of the 2017 ACM on Conference on Information and Knowledge Management*, pages 377–386, 2017.

[24] Aditya Tandon, Aiiad Albeshri, Vijey Thayananthan, Wadee Alhalabi, Filippo Radicchi, and Santo Fortunato. Community detection in networks using graph embeddings. *Physical Review E*, 103(2):022316, 2021.

[25] Scott White and Padhraic Smyth. A spectral clustering approach to finding communities in graphs. In *Proceedings of the 2005 SIAM international conference on data mining*, pages 274–285. SIAM, 2005.

[26] Vincent A Traag, Ludo Waltman, and Nees Jan Van Eck. From louvain to leiden: guaranteeing well-connected communities. *Scientific reports*, 9(1):5233, 2019.

[27] Andrew Ng, Michael Jordan, and Yair Weiss. On spectral clustering: Analysis and an algorithm. *Advances in neural information processing systems*, 14, 2001.